C++ ссылки (references)

Декларация ссылок

ссылка в C++ это альтернативное имя для уже существующей переменной

☞ декларируется в виде:

```
Type & RefName // Type - тип переменной int, double, ...
               // RefName - имя ссылки, ссылочной переменной
```

Пример

```
int i = 1;              // обычная переменная i
int& ref_i = i;         // ref_i - ссылка на i
ref_i = 8;              // изменение i через ссылку
cout << i << endl;      // 8
```

Важно помнить:

- Ссылка не может существовать сама по себе, она обязательно связана с уже существующей переменной, возможно анонимной

```
int& ref_i = i;        // OK!
int& ref_j;            // error: 'ref_j' not initialized
int& ref_j = 1;        // error: can't bind a reference to constant
const int& ref_j = 1; // reference to a constant anonymous variable
```

- Невозможно «перенаправить» ссылку на другую переменную

```
int& ref_i = i;
ref_i = j; // меняем значение переменной 'i'
```

- Адрес ссылки вернет адрес связанной с ней переменной

```
cout << &i << " AND " << &ref_i << endl;
// 0x7fffc3a1d5e8 AND 0x7fffc3a1d5e8
```

Ссылки в аргументах функции

☞ Сравните передачу переменной в функцию по ссылке и по адресу

ссылка как аргумент функции

```
void f_ref(int& x) {
  x = 1;
}
int main () {
  int z = 0;
  f_ref(z); // 'x' ссылка на 'z'
  cout << "f_ref: z= "
       << z << endl;
}
```

```
f_ref: z= 1
```

указатель как аргумент функции

```
void f_ptr(int* x) {
  *x = 1;
}
int main () {
  int z = 0;
  f_ptr(&z); // 'x' адрес 'z'
  cout << "f_ptr: z= "
       << z << endl;
}
```

```
f_ptr: z= 1
```

Пример: function swap() in C++ style with references

```
void swap(int& a, int& b) {
  int temp = a;
  a = b;
  b = temp;
}

int main() {
  int i = 2;
  int j = 40;
  cout << "before: i= "<<i<<", j= "<<j<<endl; // before: i= 2, j= 40
  swap(i,j);
  cout << "after: i= "<<i<<", j= "<<j<<endl; // after: i= 40, j= 2
}
```

Функции возвращающие ссылки

```
int& fun(int& x) {
  return x;
}
int z = 0;
fun(z) = 2; // вызов функции слева!
cout << "fun: z= " << z << endl; // fun: z= 2
```

Зачем это нужно?

- Вызов функции теперь может стоять слева: `L-value`
- Задание цепочек вызовов подобно: `cout « i « j « endl;`
- Переопределение некоторых операторов невозможно без возврата ссылки

Цепочка вызовов: `cout « i « j ...`

1. Запись `cout « i` означает вызов функции `operator«(cout,i)`
2. Цепочка `cout « i « j;` разворачивается: `operator«((cout«i),j)` и далее `operator«(operator«(cout,i),j)`
3. Чтобы цепочка работала, функция `operator«(cout,something)` должна вернуть ссылку на `cout`

Перегрузка оператора вывода

```
ostream& operator << (ostream& out, const Rational& r) {
  return out << r.x << "/" << r.y;
}
```

- ☞ Первый аргумент и возвращаемое значение имеют тип `ostream&`
- ☞ Второй аргумент тоже ссылка: это «экономит» копирование при передаче класса

Символы * и & в зависимости от контекста

1. В декларациях являются частью идентификации типа

```
int & ref_i = i; // декларируют ref_i как ссылку
int * ptr_i = 0; // декларируют ptr_i как указатель
```

2. В выражениях это операторы

```
ptr_i = &i; // взятия адреса переменной i
j = *ptr_i; // косвенный доступ к данным по адресу в ptr_i
```

Ссылки или указатели, что лучше?

- ☞ *C++ FAQ: используйте ссылки где можете, а указатели только там где должны*
- ☞ Ссылки на временные или динамические объекты порождают проблемы

пример №1: возвращаем большой объект по ссылке

```
vector<int>& BadReferenceFun() {      // very bad function
   vector<int> v {1,2,3};
   return v;
}
vector<int>& vi = BadReferenceFun(); // reference is invalid
cerr << " vi[0]= " << vi[0] << endl; // vi[0]= Segmentation fault
```

- ☞ Проблема функций, возвращающих большие объекты, решается с помощью концепции перемещений (move) в C++11

пример №2

```
vector<int>& ProblematicReferenceFun() { // questionable function
   vector<int>* v = new vector<int> {1,2,3};
   return *v;
}

vector<int>& rv = ProblematicReferenceFun(); // ok!
cerr << " rv[0]= " << rv[0] << endl;         // rv[0]= 1
delete &rv;  // ok, but no 'NULL'-reference
cerr << " rv[1]= " << rv[1] << endl;         // rv[1]= 0?

vector<int> vv = ProblematicReferenceFun();  // memory leak!
```

☞ Пользуйтесь тем средством работу которого лучше понимаете

R-value references in C++11

- R-value ссылка ссылается всегда на что-то, что может стоять только справа (R-value), но позволяет модифицировать его значение

```
int i {1}, j {2}; // 'i' and 'j' are lvalues of type 'int'
int&& rri = i;    // ERROR: cannot bind to lvalue
int&& rref = i+j; // 'i+j' is the R-value
rref += 1;        // OK rref=4
```

- «Обычные» ссылки теперь называют L-value references

```
int& li = i;            // lvalue reference
const int& lref = i+j; // lvalue const reference to rvalue
lref += 1; // ERROR: read-only reference 'lref'
```

Функции с аргументами определенными как L- и R- ссылки

```
void fun(int& x)  {   // L-reference
   cout << " fun(int&) x= " << x << endl;
}
void fun(int&& x) {   // R-reference
   cout << " fun(int&&) x= " << x << endl;
}
int i {1}, j {2};
fun(i);              // fun(int&) x= 1
fun(i+j);            // fun(int&&) x= 3
fun(std::move(i));   // fun(int&&) x= 1
```

☞ `std::move()` – функция стандартной библиотеки которая выполняет преобразование типа к R-value

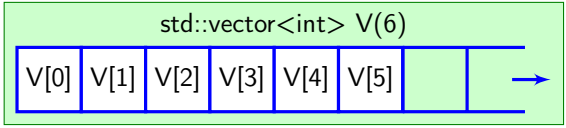
Understand auto type deduction (автоопределение типа)

```
auto x = 27;           // int (27 is int)
const auto cx = x;     // const int
auto& lrx = x;         // int &
const auto& clrx = x; // const int &

// forwarding (universal) references:
// auto&& x = ... almost same as  template<class T> T&& x = ...
auto&& ur1 = x;        // int & (x is lvalue)
auto&& ur2 = cx;       // const int & (cx is lvalue)
auto&& ur3 = 27;       // int && (27 is rvalue)
auto&& ur4 = lrx;      // int &
auto&& ur5 = cx+1;     // int &&
```

STL vector<>

Вектор (vector) это *контейнер хранящий последовательность элементов*



Вектор как динамический массив

- Тип элементов указывается в угловых скобках:
  `vector<int>, vector<double>, ...`
- Прямой доступ к произвольному элементу: `v[5] == *(&v[0]+5)`, подобно обычному массиву
- При увеличении размера вектора происходит автоматическое перераспределение памяти

STL vector: конструкторы

```
#include <vector>
using namespace std;
```

- Пустой вектор:
  `vector<int> v;`
- Вектор из N элементов «по умолчанию» или заданным значением
  ```
  vector<Rational> R(50);  // 50 elements, default initialization
  vector<double> d(10,1.); // 10 elements, initialize with 1.
  ```
- Вектор копия другого вектора:
  `vector<double> c(d);`
- В C++11 можно инициализовать списком:
  `vector<int> v {1,2,3,4,5};`

STL vector: доступ к элементам

`V[idx]`	элемент с индексом `idx`, как в обычном массиве
`V.front()`	первый элемент
`V.back()`	последний элемент
`V.at(idx)`	`== V[idx]` плюс проверка индекса (exception)

```
vector<int> v {1,2,3,4,5};
cout << " v[2]= " << v[2] << endl;                    // v[2]= 3
cout << " v first element: " << v.front() << endl;// v first element: 1
cout << " v last element: " << v.back() << endl;  // v last element: 5
cerr << " v.at(5)= " << v.at(5) << endl;
 v.at(5)= terminate called after throwing an instance of
 'std::out_of_range' what(): vector::_M_range_check:
  __n (which is 5) >= this->size() (which is 5)
```

STL vector: размер и ёмкость

- Вектор содержит информацию о числе элементов

`V.size()`	количество элементов (размер) вектора
`V.empty()`	возвращает `true` для пустого вектора
`V.resize(num)`	делает число элементов равным `num`, удаляя или добавляя из/в конец вектора
`V.resize(num,elem)`	при добавлении используется `elem`

- Можно зарезервировать область памяти для использования

`V.reserve(num)`	резервирование памяти для `num` элементов
`V.capacity()`	размер зарезервированной памяти
`V.shrink_to_fit()`	уменьшает `capacity` до `size()`

STL vector: вставка и удаление

`V.push_back(elem)`	добавляет элемент в конец вектора
`V.pop_back()`	удаляет последний элемент
`V.insert(pos,elem)`	вставляет элемент перед итератором `pos` и возвращает итератор на него
`V.insert(pos,beg,end)`	вставляет перед итератором `pos` элементы из диапазона `[beg,end)`
`V.clear()`	удаляет все элементы
`V.erase(pos)`	удаляет элемент на позиции итератора `pos` и возвращает позицию следующего элемента
`V.erase(beg,end)`	удаляет все элементы в диапазоне `[beg,end)` и возвращает позицию следующего элемента

Пример: размер и ёмкость

```
// 1) create vector of 5 elements
vector<int> vec {1,2,3,4,5}; // 1 2 3 4 5 : size= 5 capacity= 5
// 2) append one element
vec.push_back(-1);           // 1 2 3 4 5 -1 : size= 6 capacity= 10
// 3) remove the last element
vec.pop_back();              // 1 2 3 4 5  : size= 5 capacity= 10
// 4) remove all elements
vec.clear();                 //  : size= 0 capacity= 10
// 5) resize vector
vec.resize(4,0);             // 0 0 0 0  : size= 4 capacity= 10
// 6) shrink_to_fit
vec.shrink_to_fit();         // 0 0 0 0  : size= 4 capacity= 4
// 7) reserve memory
vec.reserve(25);             // 0 0 0 0  : size= 4 capacity= 25
```

Передача вектора в функцию

Пример: сумма элементов vector<int>

```
int Sum(const vector<int> & m) { // use reference!
  int s = 0;
  for ( size_t i = 0; i < m.size(); ++i ) {
      s += m[i];
  }
  return s;
}
...
vector<int> V {5,4,3,2,1};
cout << " Sum(V)= " << Sum(V) << endl; // Sum(V)= 15
```

- ☞ передавайте vector<> в функцию «по ссылке»
- ☞ константная ссылка гарантирует, что функция не меняет вектор

Использование вектора как обычного массива

- Имеется функция для работы с обычными массивами типа `int`:

```
int Sum(int m[], int n) {
  int s = 0;
  for(int i = 0; i < n; ++i) { s += m[i]; }
  return s;
}
```

Можно ли её использовать для `vector<int>`?

☞ Вектор всегда можно использовать вместо массива используя начало «внутреннего массива вектора»:

`VectorName.data()` (C++11) или `&VectorName[0]` (C++98)

```
vector<int> V {5,4,3,2,1};
cout << " Sum[V]= " << Sum(V.data(),V.size()) << endl; // Sum[V]= 15
// cout << " Sum[V]= " << Sum(&V[0],V.size()) << endl; // C++98
```

Цикл «range-for» in C++11

перебор всех элементов вектора

```
vector<int> v {1,2,3,4};
int s = 0;
for(auto x : v) { // x - это копия элемента v[i]
   s += x;
}
cout << " s= " << s << endl; // s= 10
```

перебор с изменением элементов вектора

```
s = 0;
for(auto& x : v) { // x - это ссылка на элемент v[i]
   s += x;
   x = s;          // сохраняем кумулятивную сумму
}
// v= 1 3 6 10
```

Часто используют следующий синтаксис:

1. `for(auto x : v) {...} // x - копия элемента вектора`
2. `for(auto& x : v) {...} // x - ссылка на элемент`
3. `for(const auto& x : v) {...} // x - константная ссылка`

Пример: «range-for» не только для вектора

```
int a[] {1,6,8,9};
for(const auto& x : a) { // an array
   cout << x << " ";
}
cout << endl; //  1 6 8 9
for(auto x : {1.,1.3,1.21}) { // braced-init-list
   cout << x << " ";
}
cout << endl; //  1 1.3 1.21
```

Дополнительные слайды

Массив переменной длины отсутствует в C++

Variable Length Arrays in C99

```
void fun(int n) {
  int vla[n]; // ошибка для многих компиляторов C++
  ...
}
```

В C++ вместо VLA используйте vector<>

```
#include <vector>
void fun(int n) {
  vector<int> v(n);
  ...
}
```